Week 4: BLAS 1 Optimization & SGEMM Parallel Scheduling

High-Performance Computing Project

April 17, 2026

What is BLAS Level 1?

BLAS Level 1 operations are fundamental vector math tools used inside larger algorithms (like SGEMM). They deal entirely with 1D arrays (vectors).

- **sscal**: Scales a vector by a constant ($x \leftarrow \alpha x$)
- **scopy**: Copies one vector to another ($y \leftarrow x$)
- **sswap**: Swaps the contents of two vectors
- **saxpy**: Alpha X Plus Y ($y \leftarrow \alpha x + y$)
- **sdot**: Computes the dot product of two vectors
- **snrm2**: Calculates the Euclidean norm of a vector (length)
- **sasum**: Calculates the sum of absolute values
- **isamax**: Finds the index of the maximum absolute value

Step 1: The Scalar Baseline (The "Naive" Approach)

Concept: A simple C `for` loop processing one array element at a time in sequence.

Why it fails: Processing one float per clock cycle starves the CPU. It explicitly ignores the massive arithmetic logic units (ALUs) and memory buses available on modern chips.

Results (N=1M):

Kernel	Metric	Scalar	OpenBLAS	% of OpenBLAS
`scopy`	GB/s	21.78	17.95	121%
`saxpy`	GFLOP/s	5.54	41.72	**13%**
`sdot`	GFLOP/s	1.68	7.51	**22%**
`snrm2`	GFLOP/s	1.70	5.55	**30%**

Takeaway: Simple memory moves do okay due to the CPU's hardware prefetcher predicting the linear read, but compute-heavy reductions (`sdot`, `snrm2`) are severely bottlenecked.

Step 2: SIMD Vectorization (AVX2)

Concept: Using AVX2 intrinsics (compiler functions) to load, process, and store 8 floating points simultaneously per clock cycle using 256-bit registers.

Why performance changes: We parallelize the math physically instead of logically, allowing the processor to execute 8 multiplications simultaneously (using FMA).

Results (Speedup over Scalar):

Kernel	Scalar	AVX2	Speedup
`scopy` (GB/s)	21.78	22.10	1.01x
`saxpy` (GF/s)	5.54	5.56	1.00x
`sdot` (GF/s)	1.68	7.32	**4.35x**
`snrm2` (GF/s)	1.70	14.99	**8.81x**

Takeaway: Massive speedups for reductions where math is the bottleneck. `scopy` and `saxpy` see no gain because a single core is already reading from RAM as fast as physically possible.

Step 3: Multi-threading & Latency Hiding

Concept 1 (Threading): OpenMP slices the massive array across 8 separate CPU cores. This allows us to use 8 memory channels at once.
Concept 2 (Latency Hiding): Using multiple independent accumulator registers prevents the CPU from waiting for a previous instruction to finish.

Final Results (N=1M, 8 Threads):

Kernel	AVX2	AVX2+OMP	OpenBLAS	% of OpenBLAS
`sscal` (GB/s)	19.78	**134.40**	18.12	**741%**
`saxpy` (GF/s)	5.56	**42.63**	41.72	**102%**
`sdot` (GF/s)	7.32	**54.60**	7.51	**727%**
`snrm2` (GF/s)	14.99	**91.98**	5.55	**1658%**

Takeaway: By distributing the load and keeping hardware fully saturated, we drastically outperform standard OpenBLAS implementations!

SGEMM Parallel Tasking: Loops vs. Tasks

After mastering micro-optimizations, we analyzed **how** to distribute macro workloads for Matrix Multiplication (SGEMM).

1. **Loop-based (`#pragma omp for collapse`):**
   - Like a rigid, heavily organized factory assembly line.
   - Statically or dynamically divides the matrix grid upfront.
   - **Pros:** Near-zero overhead.
2. **Task-based (`#pragma omp task`):**
   - Like a flexible restaurant kitchen ticket system.
   - A master thread dynamically spawns independent tasks (tiles). Any idle worker can take the next task packet.
   - **Pros:** Perfect for unbalanced or irregular workloads.

SGEMM Scheduling: Measuring the Overhead

Results (N=128 to 2048, 8 threads):

Matrix	LOOP (GF/s)	TASK (GF/s)	Relative Difference
128	**85.16**	6.09	**Task is 14.0x Slower**
256	**208.03**	26.06	**Task is 8.0x Slower**
512	**341.91**	204.30	**Task is 1.6x Slower**
1024	**420.54**	378.00	**Task is 1.1x Slower**
2048	**452.23**	435.48	**Task is 1.03x Slower**

Why the extreme gap for small matrices?

For small matrices, creating the task structures in memory (pointers, captured variables) takes longer than multiplying the actual tile. The CPU spends all its time "doing paperwork."

SGEMM Scheduling: The Convergence

Finding the Crossover Point:
Matrix multiplication computational work scales as $\mathcal{O}(N^3)$. However, the number of tiles (and therefore the number of tasks) only scales as $\mathcal{O}(N^2)$.

- As N doubles, the math workload multiplies by $8x$.
- As N doubles, the scheduling paperwork multiplies by only $4x$.

Eventually, the time spent inside the 6×16 micro-kernel completely overshadows the time it took to create the task. This is why at $N = 2048$, the Task-based implementation reaches 96% of the efficiency of the Loop-based implementation.

Final Comparison & Conclusion

When to use what?

- **Data Size:** Micro-parallelism (AVX2) improves performance everywhere, but Macro-parallelism (OpenMP) only delivers true speedups on large data sets where overhead is amortized.
- **Scheduling:** For highly predictable, dense data grids (like SGEMM), avoid `#pragma omp task` due to initialization overhead. Stick to `#pragma omp for`.

Week 4 Achievements:

- Built 9 BLAS Level 1 kernels from scratch with Fortran ABI compatibility.
- Decisively outperformed OpenBLAS (up to 16x) on reductions using adaptive multi-threading and latency hiding strategies.
- Quantified the exact point where dynamic task scheduling becomes viable vs. detrimental in strict mathematical workloads.